

NÚCLEO DE PESQUISA: RISCOS DA INTELIGÊNCIA ARTIFICIAL GENERATIVA NA ENGENHARIA DE SOFTWARE

Esta pesquisa acadêmica aprofunda a identificação, classificação e fundamentação empírica dos riscos inerentes à adoção de Inteligência Artificial Generativa no desenvolvimento de software, consolidando a literatura de 2021 a 2026.

A. DELIMITAÇÃO

A pesquisa mapeia as falhas sistêmicas, vulnerabilidades de segurança e vieses cognitivos (como a confiança excessiva) introduzidos pela IA Generativa no ciclo de vida do software. O escopo foca em diferenciar riscos empiricamente comprovados em estudos de Engenharia de Software e Interação Humano-Computador (IHC) de riscos teóricos baseados em projeções legais, visando estabelecer mecanismos de mitigação aplicáveis ao desenvolvimento.

B. PERGUNTAS DE PESQUISA

1. Quais são as evidências científicas que comprovam a ocorrência de alucinações, vulnerabilidades de código e perda de compreensão arquitetural induzidas por LLMs?
2. Como a literatura diferencia os riscos empíricos de operação dos riscos teóricos de propriedade intelectual e privacidade?
3. Quais são os principais riscos da utilização de IA generativa na Engenharia de Software e quais mecanismos de controle são sustentados pela literatura para integrá-los a uma metodologia de trabalho?

C. SUBTEMAS INVESTIGADOS

- Riscos Funcionais e Sintáticos: Alucinações, código incorreto/incompleto, bibliotecas e APIs inexistentes, erros silenciosos e geração de testes inadequados.
- Riscos de Segurança: Vulnerabilidades injetadas, código inseguro e exposição de informações.
- Riscos Cognitivos e Humanos: Dependência excessiva (over-reliance), automação

complacente, perda de compreensão do código e falhas de revisão.

- Riscos Sistêmicos e de Governança: Problemas arquiteturais, inconsistência entre componentes, propagação de erros, vieses, violação de propriedade intelectual (PI) e privacidade.

D. ESTRATÉGIA E CRITÉRIOS DE SELEÇÃO

Foram revisados estudos empíricos, experimentos controlados e revisões sistemáticas da literatura (SLRs) publicados entre 2021 e 2026. Priorizou-se pesquisadores reconhecidos em segurança da informação (USENIX Security, ACM SIGSAC) e Engenharia de Software (IEEE, ICSE). Critério de exclusão: artigos de opinião, blogs corporativos e conjecturas sem amparo metodológico ou legal.

E E F. MAPEAMENTO DE RISCOS E EVIDÊNCIAS

Risco 1: Alucinações, Bibliotecas Inexistentes e APIs Incorretas

- Definição: Geração de trechos de código com sintaxe válida, mas que invocam métodos, pacotes ou APIs que não existem.
- Mecanismo: Ocorre devido à natureza estocástica dos LLMs (previsão do próximo token), que interpolam nomes plausíveis que nunca foram programados no mundo real.
- Evidência Científica: HOU et al. (2023) e FAN et al. (2023) catalogam empiricamente a alucinação de APIs como a principal limitação dos modelos atuais.
- Impacto: Falha de compilação, ou pior, "Ataques de Alucinação" (onde hackers registram os nomes falsos sugeridos pela IA no repositório npm/PyPI para injetar malware).
- Atividade afetada: Programação e integração.
- Mitigação: Uso de Linters rígidos, análise estática obrigatória e proibição de importação cega.

Risco 2: Código Inseguro e Vulnerabilidades (Empiricamente Comprovado)

- Definição: Produção de código que contém falhas clássicas de segurança (ex: SQL Injection, Buffer Overflow, Hardcoded Secrets).

- Mecanismo: Os modelos são treinados em repositórios abertos (como o GitHub) que contêm historicamente enormes volumes de códigos inseguros ou desatualizados.
- Evidência Científica: PERRY et al. (2022) provaram em estudo controlado que desenvolvedores assistidos por IA escreveram código significativamente menos seguro do que o grupo de controle.
- Impacto / Probabilidade: Alta probabilidade em tarefas de criptografia e acesso a banco de dados. Impacto catastrófico (vazamento de dados).
- Atividade afetada: Programação, Arquitetura.
- Mitigação: Pipelines de CI/CD com ferramentas de SAST (Static Application Security Testing) e Code Review humano direcionado à segurança.

Risco 3: Dependência Excessiva (Over-reliance) e Erros Silenciosos

- Definição: O viés cognitivo onde o desenvolvedor assume que a máquina está correta, diminuindo o rigor analítico e aprovando falhas lógicas (erros silenciosos).
- Mecanismo: A fadiga cognitiva (VAITHILINGAM et al., 2022; SANDOVAL et al., 2023). Revisar código alheio e prolixo gera automação complacente.
- Evidência Científica: SANDOVAL et al. (2023) demonstraram que a revisão superficial não detecta vulnerabilidades inseridas por IA, e CRESCENCIO FERREIRA (2026) relata a perda de conhecimento de base.
- Impacto: Dívida técnica exponencial e perda de compreensão da arquitetura do software a longo prazo.
- Atividade afetada: Revisão de Código, Debugging, Manutenção.
- Mitigação: Adoção do modelo Human-in-the-Loop com Code Review assíncrono (longe da sugestão em tempo real da IDE) e rodízio de revisores.

Risco 4: Privacidade e Propriedade Intelectual (Teórico e Normativo)

- Definição: Exposição de segredos de negócio ou dados pessoais (LGPD) e incorporação de código protegido por licenças restritivas (GPL) sem atribuição.
- Mecanismo: Desenvolvedores colam trechos proprietários no prompt de LLMs públicos, cujos dados podem ser reaproveitados em treinamentos futuros.
- Evidência Científica: Projetado pela legislação (EU AI Act, PL 2338/23) e debatido em

literatura sobre governança corporativa.

- Impacto: Sanções legais, multas, processos de violação de copyright.
- Atividade afetada: Uso de prompts em qualquer fase.
- Mitigação: Uso exclusivo de instâncias de LLM corporativas isoladas (Enterprise) e sanitização de dados no prompt.

G. ANÁLISE CRÍTICA (CLASSIFICAÇÃO DOS RISCOS)

A literatura diferencia claramente as categorias de risco:

1. Riscos Comprovados Empiricamente: Alucinações de APIs, aumento da inserção de vulnerabilidades de segurança sob pressão (automation bias) e o paradoxo do tempo de debugging (onde a IA acelera a escrita, mas atrasa a correção de erros lógicos sutis).
2. Riscos Teoricamente Discutidos (Alta Criticidade): Propagação de erros arquiteturais complexos e violação de licenças open-source (copyleft), que dependem de jurisprudência futura.
3. Riscos baseados em Recomendação de Especialistas: Geração de testes inadequados (onde a IA escreve um teste que sempre passa, validando um código errado de forma tautológica), exigindo metodologias como TDD (Test-Driven Development) rigoroso liderado pelo humano.

H. COMPOSIÇÃO ACADÊMICA ÚNICA

A utilização da Inteligência Artificial Generativa na Engenharia de Software carrega riscos multiformes que transcendem o mero erro de sintaxe, adentrando falhas cognitivas, arquiteturais e de governança. De acordo com a literatura empírica atual, o principal risco da IA generativa não é a ausência de geração de código, mas a fabricação de 'erros silenciosos' — trechos lógicos incorretos ou inseguros embalados em uma sintaxe impecável e altamente plausível. O fenômeno das alucinações frequentemente resulta na invenção de bibliotecas ou métodos que não existem (HOU et al., 2023), abrindo vetores para ataques na cadeia de suprimentos.

De forma mais alarmante, a introdução dessas ferramentas catalisa vulnerabilidades humanas. Estudos controlados (PERRY et al., 2022; SANDOVAL et al.,

2023) comprovaram o risco da dependência excessiva (over-reliance): desenvolvedores assistidos por IA tendem a escrever códigos com mais falhas de segurança do que aqueles que desenvolvem manualmente, ironicamente acreditando que seus artefatos estão mais protegidos. Essa complacência da automação reduz drasticamente a eficácia da revisão de código (Code Review), gerando um ciclo de degradação onde a equipe perde progressivamente a compreensão da arquitetura sistêmica do projeto.

Adicionalmente, os riscos extrapolam a IDE do programador, atingindo a conformidade regulatória. O vazamento de informações confidenciais ou a violação de privacidade ao inserir dados reais em prompts configura uma quebra frontal da LGPD e do EU AI Act. Diante desse panorama, a literatura sustenta que a mitigação exige a rejeição do desenvolvimento totalmente autônomo em prol de mecanismos rigorosos de controle: adoção inegociável de Human-in-the-Loop (HITL), uso intensivo de análise estática de segurança (SAST), isolamento do contexto via Prompt Engineering estruturado e a rastreabilidade completa das intervenções algorítmicas.

I. IMPLICAÇÕES PARA A METODOLOGIA DA TG

Para o desenvolvimento do sistema de grade horária da FATEC, os riscos levantados impõem salvaguardas experimentais rigorosas:

1. Mitigação de Código Inseguro e Alucinações (Revisão Tripla): Nenhum script de banco de dados (MySQL) ou regra de matrícula gerada pela IA pode ir para a ramificação principal (main/master) sem passar por testes unitários escritos pelo próprio grupo, validando a lógica (evitando testes gerados pela IA que apenas espelhem o código falho).

2. Mitigação de Over-reliance (Supervisão Ativa): O grupo instituirá o Code Review como barreira obrigatória. O autor do PR deve explicar a lógica gerada pela IA para outro membro do grupo. Se o autor não souber explicar *como* o código funciona, ele deve ser descartado.

3. Mitigação de Privacidade: Conforme os princípios de governança, é vetado o uso de dados reais de alunos da FATEC ou arquiteturas internas sensíveis nos prompts do Gemini ou Copilot.

4. Rastreabilidade: O grupo documentará na TG quais seções do sistema sofreram maior incidência de 'erros silenciosos' gerados pela IA, comprovando a avaliação metodológica e a detecção do erro.

J. REFERÊNCIAS ABNT COMPLETAS

CRESCENCIO FERREIRA, J. V.; ROLIM DE SOUSA, R. O Impacto da Inteligência Artificial Generativa no Papel do Desenvolvedor de Software. RECIMA21 - Revista Científica Multidisciplinar, [S. l.], v. 7, n. 5, p. e757873, maio 2026. DOI: 10.47820/recima21.v7i5.7873.

FAN, X. et al. Large Language Models for Software Engineering: Survey and Open Problems. Em: International Conference on Software Engineering (ICSE). [S. l.]: IEEE/ACM, 2023. Disponível em: <https://arxiv.org/abs/2310.03533>. Acesso em: 6 out. 2026.

HOU, X. et al. Large Language Models for Software Engineering: A Systematic Literature Review. ACM Transactions on Software Engineering and Methodology (TOSEM), Nova York, v. preprint, 2023. Disponível em: <https://arxiv.org/abs/2308.10620>. Acesso em: 6 out. 2026.

PERRY, N. et al. Do Users Write More Insecure Code with AI Assistants? Em: Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security. [S. l.]: ACM, 2022.

SANDOVAL, G. et al. Security Implications of Large Language Model Code Assistants: A User Study. Em: USENIX Security Symposium. [S. l.]: USENIX Association, 2023.

VAITHILINGAM, P.; ZHANG, T.; GLASSMAN, E. L. Expectation vs. Experience: Evaluating the Usability of Code Generation Tools Powered by Large Language Models. Em: CHI Conference on Human Factors in Computing Systems. Nova York: ACM, 2022. DOI: 10.1145/3491101.3519665.

K. LACUNAS DE INVESTIGAÇÃO

- A literatura carece de metodologias unificadas para a detecção automatizada de testes tautológicos (onde a IA gera o teste baseada no erro do próprio código gerado, validando

a falha de segurança como se fosse uma funcionalidade).

- Há insuficiência de dados de longo prazo sobre o impacto da 'atrofia cognitiva' (perda de proficiência em arquitetura) em desenvolvedores iniciantes treinados exclusivamente na era da GenAI.